



Universidade Federal
do Rio de Janeiro
Escola Politécnica

Introdução ao Aprendizado de Máquina

Classificador de Aprovação de Crédito

Projeto e implementação de um classificador binário de risco de crédito
com pipeline de pré-processamento, seleção de atributos
e comparação de modelos, culminando em um LightGBM otimizado

Aluno: Henrique Kezen V. H. Leite

Repositório: _____

Ambiente técnico: Python 3.14 — Pandas, NumPy, Scikit-learn,
LightGBM, XGBoost, Matplotlib, Seaborn

Rio de Janeiro, 16 de julho de 2026

Sumário

Resumo	1
1 Objetivo do Projeto	1
2 Descrição dos Dados	1
2.1 Variável alvo	1
2.2 Tipos de variáveis	1
2.3 Valores faltantes	2
2.4 Variáveis constantes	2
2.5 Valores extremos	2
3 Metodologia	2
3.1 Estratégia geral	2
3.2 Duas métricas de validação distintas	2
3.3 Comparação de modelos	3
4 Pré-processamento dos Dados	3
4.1 Objetivo	3
4.2 Decisões de pré-processamento	3
4.3 Resultado	4
5 Seleção de Atributos	4
5.1 Objetivo	4
5.2 Metodologia de seleção	4
5.3 Variáveis mais importantes	4
5.4 Features raras	5
6 Modelagem e Treinamento	5
6.1 Objetivo	5
6.2 Resultados comparativos	5
6.3 Tentativa de <i>feature engineering</i> (Fase 2.2)	6
7 Otimização e Submissão	6
7.1 Objetivo	6
7.2 Escolha do modelo e base	6
7.3 Ajuste de threshold	7
7.4 Ensemble de folds	7
7.5 Ajuste de <code>n_estimators</code> — o refinamento que funcionou	7
7.6 Threshold fino no modelo final	8
8 Discussão dos Resultados	9
8.1 Por que o LightGBM superou os demais?	9
8.2 Por que a <i>feature engineering</i> não funcionou?	9

8.3	Por que mais árvores ajudaram até certo ponto?	9
8.4	Por que o ensemble de folds piorou?	9
9	Conclusão	9
10	Referências	10

Lista de Figuras

1	Variáveis mais importantes segundo a média dos ranks de ExtraTrees, informação mútua e χ^2	4
2	Comparação da acurácia dos quatro modelos em validação cruzada. As barras de erro representam ± 1 desvio-padrão entre os 5 folds. A ampla sobreposição indica que as diferenças <i>não</i> são estatisticamente distinguíveis.	5
3	Acurácia em função do número de árvores. A curva azul é a validação interna (CV, com banda de ± 1 desvio-padrão) e a laranja é o score público do Kaggle. Ambas atingem o máximo na região de 259–260 árvores, e ambas decaem acima disso — concordância que sustenta a escolha de 260 como modelo final.	8

Resumo

Este trabalho aborda a classificação binária de solicitantes de crédito como bons pagadores ou inadimplentes, utilizando um conjunto de 20.000 registros com 41 atributos. Foram testados quatro algoritmos — Regressão Logística, Random Forest, XGBoost e LightGBM — sob validação cruzada estratificada com 5 folds. Após comparação entre diferentes estratégias de pré-processamento e ajuste de hiperparâmetros, o modelo LightGBM, treinado sobre 1.853 features com 260 árvores de boosting e threshold de 0,50 com `random_state` fixado em 42, alcançou a melhor acurácia de 61,36 % no placar público do Kaggle.

1 Objetivo do Projeto

O objetivo deste trabalho é construir um classificador binário capaz de prever, a partir de dados de uma solicitação de crédito, se o solicitante será um bom pagador ou ficará inadimplente. A métrica de avaliação da competição é a **acurácia** — a porcentagem de previsões corretas sobre o total.

2 Descrição dos Dados

O conjunto de dados fornecido pela disciplina está dividido em duas partes:

Arquivo	Registros	Colunas	Uso
<code>conjunto_de_treinamento.csv</code>	20.000	42 (41 atributos + 1 alvo)	Treino e validação
<code>conjunto_de_teste.csv</code>	5.000	41 (sem alvo)	Geração de previsões para o Kaggle

2.1 Variável alvo

A variável `inadimplente` é binária (0 = bom pagador, 1 = inadimplente) e está perfeitamente balanceada: 10.000 registros de cada classe. Um classificador que sempre previsse a mesma classe teria acurácia de 50 %.

2.2 Tipos de variáveis

Tipo	Quantidade	Exemplos
Catégoricas codificadas	15	profissão, ocupação, estado civil, dia de vencimento
Catégoricas binárias	16	possui telefone residencial, possui cartão Visa, vínculo formal
Numéricas quantitativas	9	idade, renda mensal, meses na residência, patrimônio pessoal
Identificador	1	<code>id_solicitante</code> (não preditivo)

2.3 Valores faltantes

Seis colunas apresentam dados ausentes, com destaque para variáveis relacionadas ao companheiro do solicitante:

Variável	% de faltantes
grau_instrucao_companheiro	64,3 %
profissao_companheiro	57,6 %
profissao	15,5 %
ocupacao	14,9 %
meses_na_residencia	7,3 %
tipo_residencia	2,7 %

O elevado percentual de ausência nas variáveis do companheiro (64,3 % e 57,6 %) muito provavelmente codifica a inexistência de companheiro (solicitantes solteiros), de modo que o *padrão de ausência* carrega sinal — razão pela qual optou-se por tratar a ausência como categoria explícita em vez de descartá-la.

2.4 Variáveis constantes

`grau_instrucao` e `possui_telefone_celular` apresentaram um único valor em todo o conjunto de treinamento, não carregando informação discriminativa.

2.5 Valores extremos

Variáveis financeiras apresentaram outliers significativos: `renda_mensal_regular` tem mediana de 500 R\$ e máximo de 959.000 R\$; `valor_patrimonio_pessoal` tem mediana de 0 R\$ e máximo de 6.000.000 R\$. Esses valores extremos foram sinalizados mas não removidos.

3 Metodologia

3.1 Estratégia geral

O projeto foi organizado em fases sequenciais: pré-processamento dos dados, seleção de atributos, modelagem com comparação de algoritmos, e otimização do modelo vencedor. O conjunto de teste (5.000 amostras) foi mantido isolado até a etapa final de submissão, para evitar qualquer influência nas decisões de modelagem.

3.2 Duas métricas de validação distintas

Ao longo do relatório, duas grandezas diferentes são reportadas e *não devem ser confundidas*:

- **Validação interna (CV)** — acurácia média da validação cruzada estratificada de 5 folds sobre os 20.000 registros de treino. É a métrica usada para *tomar decisões* de modelagem, pois não depende do envio ao Kaggle.

- **Score público (Kaggle)** — acurácia calculada pela plataforma sobre a fração pública do conjunto de teste. Serve como *estimativa externa* de generalização, mas é sujeita ao ruído de uma amostra menor e ao risco de sobreajuste ao leaderboard.

Na validação cruzada, os 20.000 registros são divididos em 5 partes iguais; em cada rodada, 4 partes treinam o modelo e a parte restante é usada como teste. Cada registro é testado exatamente uma vez, e a acurácia reportada é a média das 5 rodadas. Todo o pré-processamento é reajustado apenas nos dados de treino de cada fold, evitando vazamento.

3.3 Comparação de modelos

- **Regressão Logística** — classificador linear utilizado como baseline de referência. Sua principal função foi estabelecer um piso de acurácia: qualquer modelo mais complexo que não supere esse piso não justifica sua complexidade adicional.
- **Random Forest** — ensemble de árvores de decisão independentes, combinadas por votação majoritária.
- **XGBoost** — algoritmo de gradient boosting que constrói árvores sequencialmente, em que cada nova árvore aprende a corrigir os erros das anteriores.
- **LightGBM** — variante do gradient boosting otimizada para velocidade e eficiência, com a mesma lógica sequencial do XGBoost.

4 Pré-processamento dos Dados

4.1 Objetivo

Transformar a base bruta em uma matriz numérica pronta para modelagem, utilizando um Pipeline do scikit-learn como fonte da verdade, garantindo reprodutibilidade e evitando vazamento de dados.

4.2 Decisões de pré-processamento

Aspecto	Decisão	Justificativa
Codificação categórica	One-Hot Encoding com <code>handle_unknown="ignore"</code>	Evita ordem falsa entre categorias e aceita categorias novas no teste
Faltantes numéricos	Imputação pela mediana	Robusta contra outliers
Faltantes categóricos	Categoria MISSING	Preserva o sinal de ausência como possível informação
Escala numérica	<code>RobustScaler</code>	Mais estável na presença de valores extremos
Outliers	Flags acima do percentil 99 + transformação <code>log1p</code>	Preserva a informação sem distorcer o modelo
Colunas removidas	<code>id_solicitante</code> , <code>grau_instrucao</code> , <code>possui_telefone_celular</code>	Identificador e variáveis constantes

4.3 Resultado

A matriz processada final ficou com **20.000 linhas** $\times$ **1.853 features**, armazenada em formato esparsa (.npz), adequado para a alta dimensionalidade gerada pelo One-Hot Encoding.

5 Seleção de Atributos

5.1 Objetivo

Analisar quais atributos contribuem para a previsão de inadimplência, identificar redundâncias e preparar conjuntos candidatos de features para a modelagem.

5.2 Metodologia de seleção

A análise combinou três abordagens complementares:

1. **Importância por árvores (ExtraTrees)** — mede quanto cada feature contribui para as decisões do modelo.
2. **Informação mútua** — mede dependência estatística não-linear entre cada feature e o alvo.
3. **Teste qui-quadrado (χ^2)** — mede associação estatística entre features categóricas e o alvo.

5.3 Variáveis mais importantes

A Figura 1 apresenta as oito variáveis mais relevantes, agregadas por variável original e ordenadas pelo rank médio das três abordagens (quanto menor o rank, mais importante).

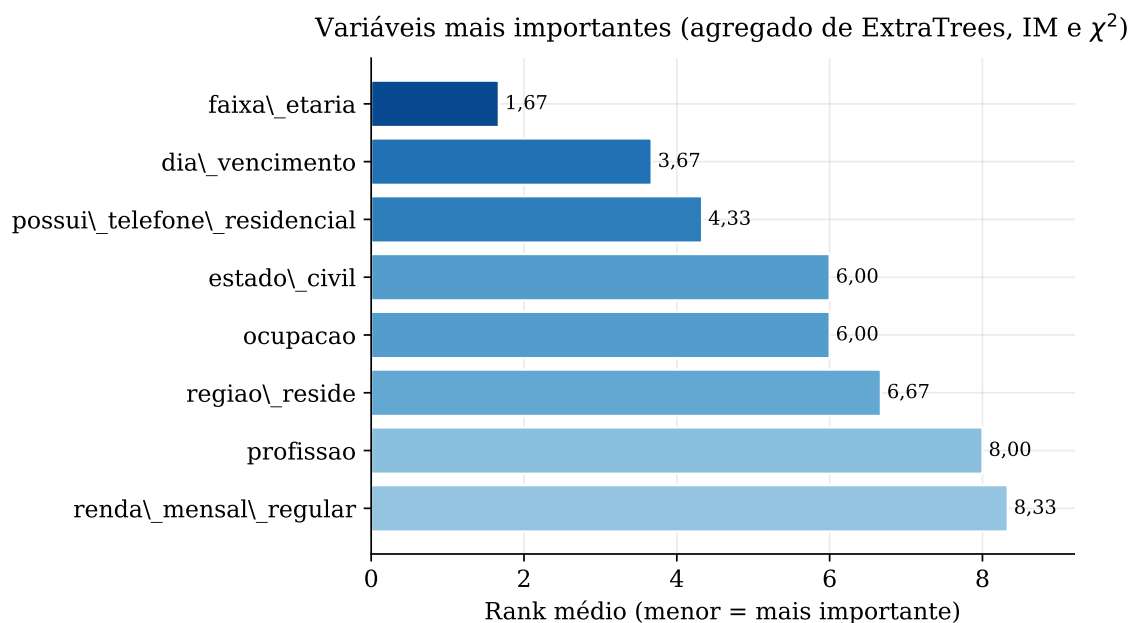


Figura 1: Variáveis mais importantes segundo a média dos ranks de ExtraTrees, informação mútua e χ^2 .

Chama atenção a presença de `dia_vencimento` e `possui_telefone_residencial` entre as variáveis mais discriminativas — atributos aparentemente operacionais e não socioeconômicos.

5.4 Features raras

44 features foram identificadas como muito raras (presença $< 1\%$ das amostras), concentradas principalmente em categorias de profissão, cartões de crédito e tipos de residência.

6 Modelagem e Treinamento

6.1 Objetivo

Comparar o desempenho dos quatro algoritmos e identificar o mais promissor para refinamento.

6.2 Resultados comparativos

Os quatro modelos foram avaliados nas mesmas condições: validação cruzada estratificada com 5 folds sobre a base de 1.853 features. A Figura 2 mostra a acurácia média com as respectivas barras de erro (um desvio-padrão entre folds).

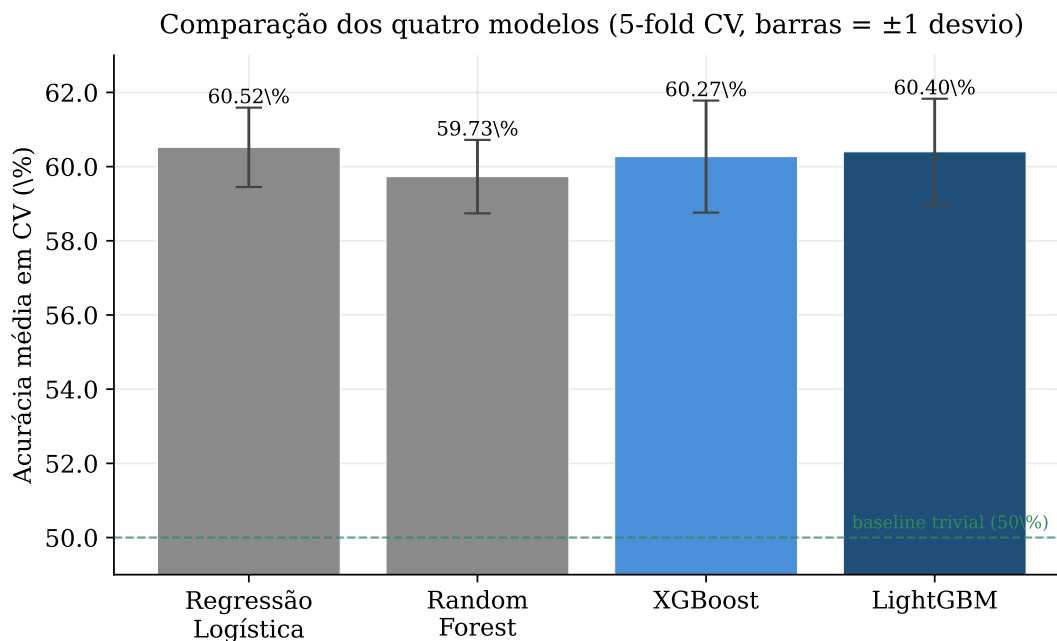


Figura 2: Comparação da acurácia dos quatro modelos em validação cruzada. As barras de erro representam ± 1 desvio-padrão entre os 5 folds. A ampla sobreposição indica que as diferenças *não* são estatisticamente distinguíveis.

Os quatro modelos ficaram numa faixa estreita de acurácia (59,7% a 60,5%), com os dois algoritmos de boosting (XGBoost e LightGBM) ligeiramente à frente. Como evidencia a sobreposição das barras de erro na Figura 2, essa diferença está dentro do ruído da validação interna. Nas submissões ao Kaggle, porém, o LightGBM superou consistentemente os demais, e foi o modelo levado adiante.

O fato de o baseline linear (regressão logística) alcançar acurácia muito próxima dos modelos não-lineares indica que o sinal discriminativo nos dados é predominantemente captável de forma simples — os modelos de árvore não encontraram interações complexas que a regressão não veja.

6.3 Tentativa de *feature engineering* (Fase 2.2)

Após os primeiros resultados, exploramos uma variação do pré-processamento focada em performance, adicionando 26 variáveis engenheiradas: razões financeiras (renda por dependente, patrimônio sobre renda), scores de contato, flags de estabilidade geográfica e faixas de idade, totalizando 1.900 features. A hipótese era que essas variáveis derivadas capturariam padrões adicionais não presentes nas features originais.

Modelo	Base	Acurácia CV	Score Kaggle
LightGBM	Original (1.853 features)	60,40 %	60,64 %
LightGBM	Engenheirada (1.900 features)	60,48 %	~58,00 %

A base engenheirada apresentou acurácia interna marginalmente superior, porém **quando submetida ao Kaggle, o resultado foi significativamente pior**: 58,00 % contra 60,64 % da base original. Isso indica que as features engenheiradas, embora parecessem promissoras na validação interna, não generalizaram para dados novos — possivelmente por overfitting às particularidades do conjunto de treinamento. Este é o exemplo mais claro no projeto de *divergência entre a validação interna e o score público*: melhora numa, piora na outra. Com base nesse resultado, a base original de 1.853 features foi mantida como definitiva.

7 Otimização e Submissão

7.1 Objetivo

Extrair o máximo de performance do modelo vencedor (LightGBM com base original) por meio de ajustes sistemáticos de hiperparâmetros.

7.2 Escolha do modelo e base

As submissões iniciais confirmaram que:

Trilha	Modelo	Score Kaggle
Base original	LightGBM (171 árvores)	60,64 %
Base original	XGBoost	59,20 %
Base engenheirada	LightGBM	~58,00 %

O LightGBM sobre a base original foi selecionado como modelo principal.

7.3 Ajuste de threshold

Variações no threshold de classificação não trouxeram ganho:

Threshold	Score
0.50	60,64 %
0.49	60,60 %
0.51	60,40 %
0.52	60,08 %

O threshold padrão de 0,50 já estava no ponto ótimo.

7.4 Ensemble de folds

Testamos a média de probabilidades de 5 modelos treinados em folds diferentes:

Estratégia	Score
Modelo único (20.000 dados)	60,64 %
Ensemble de 5 folds	60,20 %

O ensemble piorou o resultado. A explicação mais provável é que cada modelo treinou com apenas 80 % dos dados, e neste dataset a perda de informação foi maior que o ganho de estabilidade.

7.5 Ajuste de `n_estimators` — o refinamento que funcionou

Mantivemos todos os hiperparâmetros fixos e variamos apenas o número de árvores do LightGBM. Além do score público do Kaggle, todas as configurações foram avaliadas por validação cruzada estratificada de 5 folds, usando os mesmos folds (`random_state=42`) e threshold de 0,50. A Figura 3 sobrepõe as duas curvas, permitindo verificar se validação interna e Kaggle concordam.

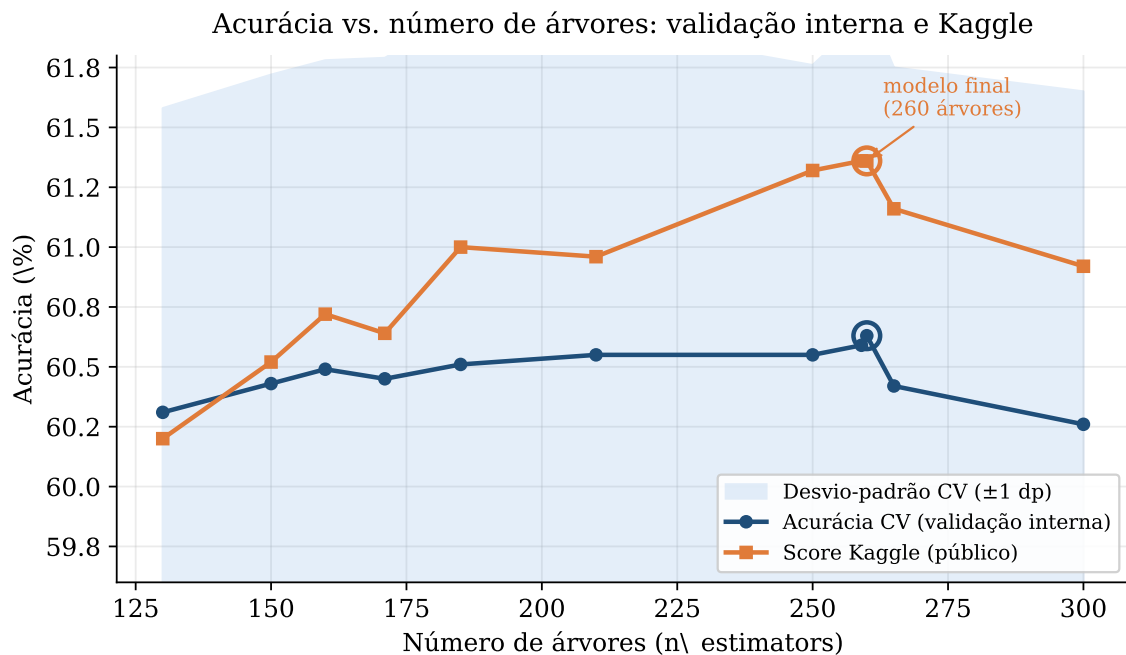


Figura 3: Acurácia em função do número de árvores. A curva azul é a validação interna (CV, com banda de ± 1 desvio-padrão) e a laranja é o score público do Kaggle. Ambas atingem o máximo na região de 259–260 árvores, e ambas decaem acima disso — concordância que sustenta a escolha de 260 como modelo final.

O modelo com 260 árvores obteve a maior acurácia média na validação interna (60,63 %) e o melhor resultado no Kaggle (61,36 %). Em relação às 171 árvores originais, o ganho foi de **+0,18 ponto percentual na validação cruzada** e de **+0,72 ponto percentual no Kaggle**. É importante notar que as médias internas ficaram muito próximas e seus desvios se sobrepõem amplamente: a validação cruzada favorece *levemente* 260 árvores, mas não demonstra superioridade estatisticamente clara. O que dá confiança à escolha é a *concordância de direção* entre as duas curvas — ambas sobem até 259 a 260 e decaem depois. A queda observada em 265 e 300 árvores é compatível com o início de overfitting, embora a variabilidade entre folds recomende cautela nessa interpretação.

7.6 Threshold fino no modelo final

Após fixar `n_estimators=260`, testamos thresholds finos que confirmaram 0,50 como ponto ótimo:

Threshold	Score
0.495	61,12 %
0.500	61,36 %
0.505	61,32 %

8 Discussão dos Resultados

8.1 Por que o LightGBM superou os demais?

Os quatro modelos testados em validação cruzada apresentaram acurácias próximas (59% a 61%), mas o LightGBM generalizou melhor nas submissões ao Kaggle, superando inclusive o XGBoost — que usa a mesma família de algoritmo. Uma possível explicação é que as diferenças de implementação entre LightGBM e XGBoost (como a forma de construir as árvores e as heurísticas de regularização) favoreceram o LightGBM para este dataset específico. Em relação ao Random Forest e à Regressão Logística, o gradient boosting captura padrões sutis que um ensemble independente ou um modelo linear não alcançam. Essa vantagem, embora pequena em validação interna, se amplificou em dados novos.

8.2 Por que a *feature engineering* não funcionou?

A base engenheirada (Fase 2.2) trouxe um resultado paradoxal: métricas internas ligeiramente superiores, mas performance real inferior no Kaggle. Isso sugere que as variáveis derivadas — razões financeiras, scores de contato, flags de estabilidade — se ajustaram a particularidades do conjunto de treinamento que não se repetiram no conjunto de teste. Em outras palavras, as features engenheiradas introduziram overfitting. A lição é que mais features não garantem melhor generalização.

8.3 Por que mais árvores ajudaram até certo ponto?

O número original de árvores (171) foi definido a partir das iterações escolhidas por early stopping nos folds de validação. Porém, o modelo final é treinado nos 20.000 registros completos, sem separar um fold de validação, e pode sustentar mais iterações. A comparação direta dos valores fixos por validação cruzada encontrou a maior média com 260 árvores (60,63%, contra 60,45% com 171), em concordância com o melhor score do Kaggle. Como essa diferença interna é pequena perante a variação entre folds, a evidência sugere — mas não prova isoladamente — uma vantagem da região de 259 a 260 árvores. A piora em 265 e 300 árvores é compatível com o início de overfitting.

8.4 Por que o ensemble de folds piorou?

A estratégia de ensemble por média de probabilidades de 5 modelos tipicamente reduz variância e melhora a robustez. No entanto, cada modelo treinou com apenas 16.000 dos 20.000 dados disponíveis. Neste dataset, a perda de 20% dos dados de treino custou mais do que o ganho de estabilidade. O modelo único treinado em todos os 20.000 registros provou ser superior.

9 Conclusão

O modelo final utilizado na competição é um **LightGBM** com a seguinte configuração:

Parâmetro	Valor
Algoritmo	LightGBM (<code>LGBMClassifier</code>)
Pré-processamento	One-Hot Encoding, imputação por mediana/ <code>MISSING</code> , <code>RobustScaler</code>
Número de features	1.853
<code>n_estimators</code>	260
<code>learning_rate</code>	0.05
<code>num_leaves</code>	31
<code>min_child_samples</code>	40
<code>subsample</code>	0.85
<code>colsample_bytree</code>	0.85
<code>reg_alpha</code>	0.1
<code>reg_lambda</code>	1.0
Threshold de classificação	0.50
<code>random_state</code>	42
Dados de treino	20.000 registros (conjunto completo)
Melhor score Kaggle	61,36 %

A estratégia que se mostrou mais efetiva foi manter o pré-processamento simples (sem feature engineering adicional), utilizar o algoritmo LightGBM, e ajustar o número de árvores além do ponto indicado pelo early stopping da validação cruzada. Tentativas de refinar threshold, construir ensembles de folds ou adicionar features engenheiradas não trouxeram ganho no placar real.

10 Referências

- Scikit-learn: <https://scikit-learn.org/>
- LightGBM: <https://lightgbm.readthedocs.io/>
- XGBoost: <https://xgboost.readthedocs.io/>
- Pandas: <https://pandas.pydata.org/>